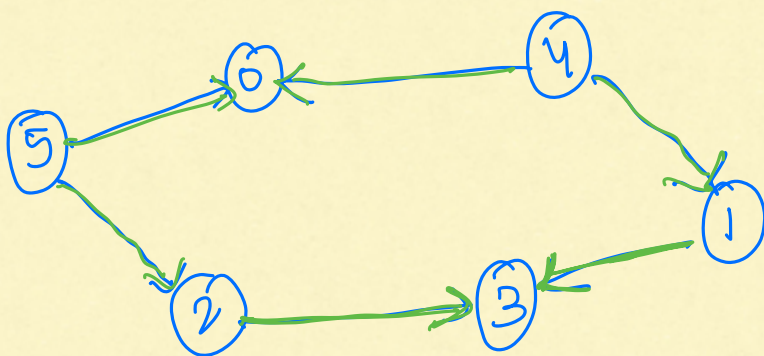


GRAPH



[[] [3] [3] [] [0, 1] [0, 2]]

idx

0

1

2

3

4

5

convert

adj = [[], [3], [3], [], [0, 1], [0, 2]]

0

1

2

3

4

5

adj-list = { 0: []

1: [3]

2: [3]

3: []

4: [0, 1]

5: [0, 2]

adj-list

{ 0: [1]
0: [] }

v: []

5-10 min

for i in range(len(adj)):

adj-list[i] = adj[i]

i = 0, 1, 2, 3

{ 0: adj[0] = []

1: adj[1] = [3]

2: adj[2] = [3]

3: adj[3] = []

(shortest path)

Dijkstra Algo

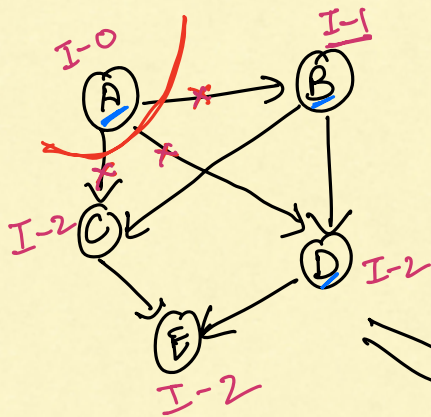
1/2 lines

(MST)

Prim's Algo

Course Schedule

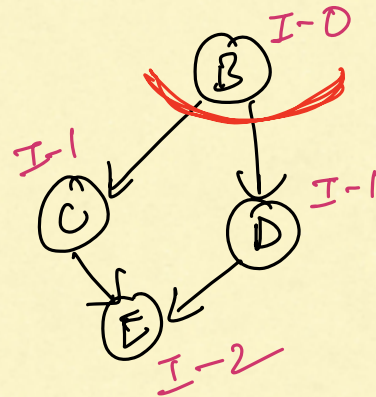
STEP-1 create adj-list



{ A: [B, C, D]
B: [C, D]
C: [E]
D: [E]
E: [] }

Indegree

A: 0
B: 1
C: 2
D: 2
E: 2



Indegree = 23

⇒ for i in adj-list:
Indegree[i] = 0

for i in adj-list:
for n in adj-list[i]
Indegree[n] = Indegree[n] + 1

queue = []

for i in indegree:
if indegree[i] == 0:
queue.append(i)

while queue != []:

queue = [A, B, C, D]

curr_node = queue.pop left()

curr_node = B

curr_node

return n
true
[A, B, C, D, E]
[A, B, D, C, E]
topo = [A]

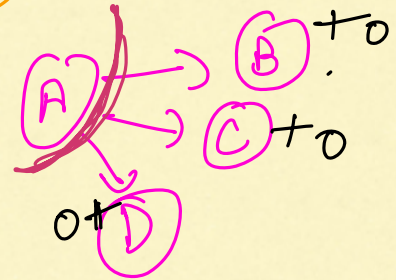
A: 0-1
B: 0

for n in adj-list[A]:
 indegree[n] = indegree[n] + 1

if indegree[n] == 0:
queue.append(n)

Indegree

A: 0
 B: ~~0~~ ~~1~~ -1
 C: ~~0~~ ~~1~~ ~~1~~ 0
 D: ~~0~~ ~~1~~ ~~1~~ 0
 E: 0 ~~1~~ ~~2~~



Prims Algo
 ↳ For MST

```

def spanningTree(self, V, adj):
    #code here
    sum=0
    adj_list,visited,distance = {},{},{}
    for i in range(V):
        adj_list[i] = []
        visited[i] = False
        distance[i] = float('inf')
    distance[0] = 0
    sum= 0

    for i in range(V):
        for (j,d) in adj[i]:
            adj_list[i].append((j,d))

    for i in range(V):
        min_distance = float('inf')
        min_distance_node = None

        #finding node with min distance
        for i in distance:
            if visited[i]==False and distance[i]<min_distance:
                min_distance = distance[i]
                min_distance_node = i
        sum = sum+min_distance
        visited[min_distance_node] = True

    for (node,dist) in adj_list[min_distance_node]:
        if visited[node]==False and dist<distance[node]:
            distance[node] = dist
    return sum

```